```
"""
nexus.core.logging
====================

Structured logging setup for NEXUS.

Emits JSON lines to a rotating file
(for audit/ingestion into log
tooling)
and human-readable text to the
console (for interactive
development).
Built entirely on the standard
library ``logging`` module — no
third-party
logging framework is required at the
core layer.

Every log record automatically
carries:
    * ``timestamp`` (UTC, ISO 8601)
    * ``level``
    * ``logger`` (module name)
    * ``message``
    * any extra structured fields
passed via ``logger.info(msg, extra=
```

```
{...})``

Usage:
    from nexus.core.logging import
configure_logging, get_logger



configure_logging(log_level="INFO",
log_dir="./data/logs")
    logger = get_logger(__name__)
    logger.info("Action approved",
extra={"action_type": "tap",
"user_id": "u_123"})
"""

from __future__ import annotations

import json
import logging
import logging.handlers
import sys
from datetime import datetime,
timezone
from pathlib import Path
from typing import Any

# Fields present on every stdlib
LogRecord that we do NOT want to
duplicate
```

```
# into the "extra" section of our
JSON output (they're either already
# emitted under a different key, or
are internal bookkeeping we don't
need).
_STANDARD_LOG_RECORD_FIELDS =
frozenset(
    logging.LogRecord(
        name="", level=0,
pathname="", lineno=0, msg="", args=
(), exc_info=None
    ).__dict__.keys()
) | {"message", "asctime"}

_CONFIGURED = False

_VALID_LEVELS = {"DEBUG", "INFO",
"WARNING", "ERROR", "CRITICAL"}


def _resolve_level(level_name: str) -
> int:
    """Translate a level name (case-
insensitive) into the ``logging``
module's int constant."""
    normalized =
level_name.strip().upper()
    if normalized not in
_VALID_LEVELS:
```

```
        valid = ",
".join(sorted(_VALID_LEVELS))
        raise ValueError(f"Invalid
log level {level_name!r}. Must be one
of: {valid}")
    return getattr(logging,
normalized)


class
JsonFormatter(logging.Formatter):
    """Renders each log record as a
single line of JSON."""

    def format(self, record:
logging.LogRecord) -> str:
        payload: dict[str, Any] = {
            "timestamp":
datetime.fromtimestamp(
                record.created,
tz=timezone.utc
            ).isoformat(),
            "level":
record.levelname,
            "logger": record.name,
            "message":
record.getMessage(),
        }
```

```
        if record.exc_info:
            payload["exception"] =
self.formatException(record.exc_info)

        # Pull in any caller-supplied
structured fields from `extra=...`.
        extras = {
            key: value
            for key, value in
record.__dict__.items()
            if key not in
_STANDARD_LOG_RECORD_FIELDS
        }
        if extras:
            payload["extra"] =
_json_safe(extras)

        return json.dumps(payload,
default=str, ensure_ascii=False)


def _json_safe(value: Any) -> Any:
    """Best-effort conversion of
arbitrary extra-field values into
JSON-safe types."""
    if isinstance(value, (str, int,
float, bool)) or value is None:
        return value
    if isinstance(value, dict):
```

```
        return {str(k): _json_safe(v)
for k, v in value.items()}
    if isinstance(value, (list,
tuple, set)):
        return [_json_safe(v) for v
in value]
    return str(value)


class
HumanReadableFormatter(logging.Format
ter):
    """Concise, colorless console
formatter for interactive
development."""

    def __init__(self) -> None:
        super().__init__(
            fmt="%(asctime)s | %
(levelname)-8s | %(name)s | %
(message)s",
            datefmt="%H:%M:%S",
        )


def configure_logging(
    *,
    log_level: str = "INFO",
    log_dir: str | Path =
```

```
"./data/logs",
    log_filename: str =
"nexus.jsonl",
    max_bytes: int = 10 * 1024 *
1024,
    backup_count: int = 5,
    enable_console: bool = True,
) -> None:
    """
    Configure the root logger for the
entire NEXUS process. Idempotent —
    calling this more than once (e.g.
from tests) will not duplicate
    handlers.

    Args:
        log_level: One of the
standard levels ("DEBUG", "INFO",
"WARNING",
            "ERROR", "CRITICAL").
        log_dir: Directory the
rotating JSON log file is written
into.
            Created if it does not
already exist.
        log_filename: Name of the
rotating log file within ``log_dir``.
        max_bytes: Rotate the file
once it exceeds this size.
```

```
        backup_count: Number of
rotated backups to retain.
        enable_console: Whether to
also attach a human-readable console
            handler (typically
disabled in headless production
deployments
            that ship only structured
JSON to a log collector).
    """
    global _CONFIGURED

    root_logger =
logging.getLogger("nexus")

root_logger.setLevel(_resolve_level(l
og_level))

    # Remove any handlers from a
previous configure_logging() call so
we
    # don't end up with duplicated
log lines if this is called again
    # (e.g. once in production
entrypoint, once in a test fixture).
    for handler in
list(root_logger.handlers):

root_logger.removeHandler(handler)
```

```
        handler.close()

    log_dir_path = Path(log_dir)
    log_dir_path.mkdir(parents=True,
exist_ok=True)

    file_handler =
logging.handlers.RotatingFileHandler(
        filename=log_dir_path /
log_filename,
        maxBytes=max_bytes,
        backupCount=backup_count,
        encoding="utf-8",
    )

file_handler.setFormatter(JsonFormatt
er())

root_logger.addHandler(file_handler)

    if enable_console:
        console_handler =
logging.StreamHandler(stream=sys.stdo
ut)

console_handler.setFormatter(HumanRea
dableFormatter())

root_logger.addHandler(console_handle
```

```
r)

    # Prevent double-propagation to
the interpreter's default root
logger,
    # which could otherwise duplicate
output if some third-party library
    # also configures
logging.basicConfig().
    root_logger.propagate = False

    _CONFIGURED = True


def get_logger(name: str) ->
logging.Logger:
    """
    Return a logger namespaced under
``nexus.<name>``. Automatically
applies
    a minimal default configuration
(console-only, INFO level) if
    ``configure_logging()`` has not
been called yet, so importing and
using
    a logger from a unit test never
crashes for lack of setup.
    """
    if not _CONFIGURED:
```

```
configure_logging(enable_console=True
)

    qualified_name = name if
name.startswith("nexus") else
f"nexus.{name}"
    return
logging.getLogger(qualified_name)
```